



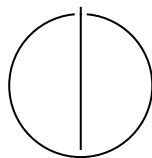
SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Reduction of Memory in Tensor Network Contractions

Fam Shihata





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

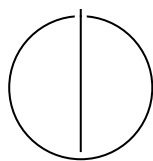
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Reduction of Memory in Tensor Network
Contractions**

**Speicherreduzierung bei
Tensornetzwerkkontraktionen**

Author:	Fam Shihata
Examiner:	Manuel Geiger
Supervisor:	Advisor
Submission Date:	Submission date



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, Submission date

Fam Shihata

Acknowledgments

Abstract

Contents

Acknowledgments	iv
Abstract	v
1 Introduction	1
1.1 Problem Formulation and Challenges	2
1.2 Research Aim	3
1.3 Contributions of This Work	3
2 Background	4
2.1 Foundations of Tensor Networks	4
2.1.1 Tensors	4
2.1.2 Tensor Contractions	4
2.1.3 Tensor Networks	5
2.2 Basic Linear Algebra Subprograms	7
2.2.1 Level 1: Vector Operations	7
2.2.2 Level 2: Matrix-Vector Operations	7
2.2.3 Level 3: Matrix-Matrix Operations	8
2.3 High-performance Matrix-Matrix Multiplication Implementations	8
2.4 High-performance Tensor Contraction Implementations	9
3 Related Work	11
3.1 Treewidth-Based and Graph-Theoretic Methods	11
3.2 Hypergraph Partitioning Approaches	11
3.3 Slicing Techniques	12
3.4 Recompression and Tensor Decomposition	13
3.5 Memory Movement and Data Locality	13
3.6 Summary and Research Gap	13
4 Implementation	16
4.1 Interface	16
4.2 Preparation and Peripherals	18
4.2.1 Persistent Contraction Path	18

Contents

4.2.2 APIs and ABIs	19
4.3 The Greedy Algorithm	20
Abbreviations	21
List of Figures	22
List of Tables	23
Bibliography	24

1 Introduction

Tensor network contraction (TNC) has become a core computational primitive in fields ranging from quantum many-body physics and quantum circuit simulation to chemistry and high-dimensional machine learning and data analysis. Their appeal comes from the ability to represent and manipulate exponentially large objects through structured factorizations, often making previously intractable problems accessible in practice [5]. Yet this promise comes with a major obstacle: the cost of contracting a tensor network can grow dramatically depending on how intermediate tensors are formed. In many realistic settings, the limiting factor is not only the number of arithmetic operations, but the peak memory required to store these intermediates. A contraction strategy that is theoretically efficient in floating-point operations (FLOPS) may still be unusable in practice if it creates tensors too large to fit in memory.

This makes memory reduction a central problem rather than a secondary implementation concern. Prior work on contraction ordering has already shown that the sequence in which tensors are contracted has a decisive impact on computational feasibility and overall performance [18, 19]. At the same time, high-performance tensor contraction frameworks and tensor-transposition libraries have demonstrated that practical efficiency depends heavily on how contractions are mapped onto hardware, how data is rearranged, and how temporary storage is managed [11, 22]. In other words, optimizing tensor networks requires attention not only to algebraic structure, but also to memory footprint, data movement, and layout-aware execution.

The importance of optimizing tensor network contractions originates from both theoretical and practical considerations. From a theoretical perspective, tensor contraction is known to be computationally hard in general, with complexity that can grow exponentially with the size of the network [18]. From a practical standpoint, modern applications such as quantum circuit simulation and many-body physics rely heavily on large-scale tensor contractions, where inefficiencies quickly become restrictive.

A key observation in high-performance computing (HPC) is that memory bandwidth and data movement increasingly dominate execution time, often surpassing the cost of FLOPS [11, 12]. In large-scale simulations, such as those targeting quantum supremacy benchmarks [23, 12], the ability to efficiently manage memory and minimize data movement is critical to achieving feasible runtimes.

Moreover, the peak memory usage during tensor contraction directly determines

whether a computation is even possible on a given system. Many tensor network contractions fail not due to computational limits, but because intermediate tensors exceed available memory. As a result, reducing peak memory usage is not merely an optimization objective, but a fundamental requirement for scalability.

Despite these challenges, most existing optimization techniques primarily focus on reducing computational cost (FLOPS) through contraction ordering, often overlooking memory-related factors such as tensor layout, data locality, and permutation overhead. This creates a gap between theoretical optimization and practical performance, particularly on modern architectures where memory hierarchy plays a central role.

1.1 Problem Formulation and Challenges

Ultimately, the goal of contraction is to compute a final tensor by successively contracting pairs of tensors along shared indices. This process involves three key components: the contraction order, which determines the sequence in which tensor pairs are combined; the intermediate tensor shapes, which define the sizes and dimensionalities of tensors generated during the contraction; and the tensor layout and permutation, which determine how tensor indices are arranged in memory and therefore how efficiently operations can be executed.

While contraction ordering has been extensively studied, the challenges associated with intermediate tensor shapes and tensor layouts remain significant and often underexplored. In particular, tensor permutations play a crucial role in enabling efficient computation. High-performance implementations typically rely on transforming tensor contractions into matrix multiplications via reshaping and permutation, a technique known as Tensor-Tensor Generalized Multiplication (TTGT) [22].

However, these permutations introduce additional memory movement, which can be costly. In many cases, tensors are repeatedly permuted and reshaped throughout the contraction sequence. This leads to increased memory bandwidth usage, poor cache locality, and higher peak memory consumption due to the creation of intermediate copies. These effects collectively degrade performance and can become a dominant bottleneck in large-scale computations.

Furthermore, existing approaches typically treat permutations as local operations required to enable individual contractions. This local perspective ignores the global structure of the contraction sequence and can result in redundant or suboptimal data transformations that accumulate over the course of execution [3].

1.2 Research Aim

This thesis proposes a complementary approach that shifts part of the optimization focus to the *post-contraction phase*. Instead of solely optimizing the contraction order, we investigate how global tensor permutations can be applied after determining a contraction path to improve memory efficiency.

The key idea is to treat tensor layout as a first-class optimization variable. By analyzing the entire contraction sequence, it becomes possible to reduce redundant permutations across consecutive contractions, improve data locality and cache utilization, and minimize peak memory usage by carefully controlling intermediate tensor layouts.

This approach is relatively niche compared to traditional methods, as most existing work focuses on pre-contraction optimization and local permutation strategies. However, given the growing importance of memory efficiency in modern computing systems, global permutation optimization presents a promising direction for further research.

1.3 Contributions of This Work

In summary, this thesis makes several key contributions. It introduces a novel framework for post-contraction optimization based on global tensor permutations. It provides a detailed analysis of the impact of tensor layouts on memory usage and data movement. It highlights the limitations of existing pre-contraction optimization techniques in addressing memory-related challenges. Finally, it demonstrates how tensor permutation strategies can complement contraction ordering to achieve improved overall performance.

By bridging the gap between contraction planning and memory-aware tensor layout optimization, this work aims to provide a more comprehensive approach to efficient tensor network contraction.

2 Background

2.1 Foundations of Tensor Networks

2.1.1 Tensors

For our discussion, it is important to define what tensors are, how they are contracted, and how they are represented in networks. A tensor is a multidimensional array that generalizes the concept of vectors and matrices to arbitrary dimensions. Formally, a tensor of order n (n -mode tensor or rank n tensor) is a collection of entries indexed by n indices, each ranging over a finite set of dimension sizes. For instance, a matrix is a second-order ($n = 2$) tensor with two indices, while a vector is a first-order ($n = 1$) tensor with a single index (see Figure 2.1). Tensors are fundamental objects in linear algebra and multilinear computations, enabling compact representations of high-dimensional data and operations on them. The elements of a tensor $\mathcal{T}$ are accessed via multi-indices, denoted $\mathcal{T}_{i_1, i_2, \dots, i_n}$, where each i_k ranges from 1 to d_k , the size of dimension k . The shape of a tensor, given by the tuple $(d_1, d_2, \dots, d_n)$, determines the total number of entries, which scales as the product of all dimension sizes. This exponential growth in storage and computation with dimension and order motivates the use of structured factorizations and tensor networks, which exploit sparsity and algebraic structure to make high-dimensional problems computationally tractable.

2.1.2 Tensor Contractions

Tensor contractions have emerged as one of the fundamental operations on tensors. At its core, a tensor contraction represents a generalized summation over shared indices between tensors. This operation constitutes many familiar linear algebra operations such as matrix multiplication, but extends naturally to higher-order interactions. Formally, a tensor contraction between two tensors $\mathcal{A}$ and $\mathcal{B}$ over a set of shared indices S is defined as:

$$C_{j_1, \dots, j_m, k_1, \dots, k_n} = \sum_{i_1=1}^{d_1} \sum_{i_2=1}^{d_2} \cdots \sum_{i_r=1}^{d_r} \mathcal{A}_{i_1, \dots, i_r, j_1, \dots, j_m} \cdot \mathcal{B}_{i_1, \dots, i_r, k_1, \dots, k_n} \quad (2.1)$$

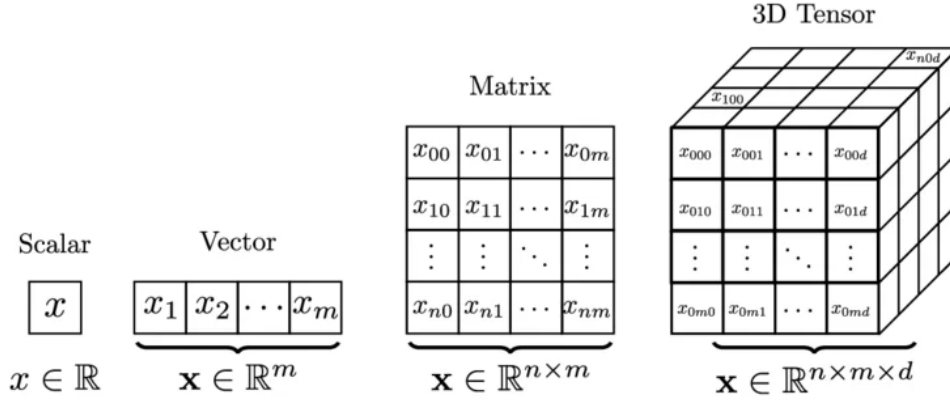


Figure 2.1: Examples of rank-0, 1, 2, 3 tensors (i.e. a scalar, vector, matrix, order-3 tensor)

where indices $i_1, \dots, i_r$ are the contracted (summed) indices shared between $\mathcal{A}$ and $\mathcal{B}$, and indices $j_1, \dots, j_m$ and $k_1, \dots, k_n$ are the free indices that remain in the result.

A more compact and standard representation of tensor contractions is given by Einstein notation (Einstein summation convention), where repeated indices are implicitly summed over. For instance, the contraction above can be written concisely as:

$$\mathcal{C}_{jk} = \mathcal{A}_{ij} \mathcal{B}_{ik} \quad (2.2)$$

where i is the repeated (contracted) index and is implicitly summed over all its dimensions, while j and k are free indices appearing in the result. This notation extends naturally to higher-order tensors and multiple contractions, making it a powerful tool for expressing complex tensor network computations in a compact form.

2.1.3 Tensor Networks

However, this is not the only way to effectively represent complex contractions of networks; we can also draw them in tensor networks. A tensor network is a graphical representation of a set of tensors and their contraction structure, where nodes represent tensors and edges represent shared indices.

In this graphical framework, each tensor is depicted as a node (typically drawn as a circle, square, or other shape), with a number of lines (legs) extending from it equal to the tensor's order. Each leg corresponds to one index of the tensor. When two legs are connected by an edge, the corresponding indices are contracted (summed over). Free

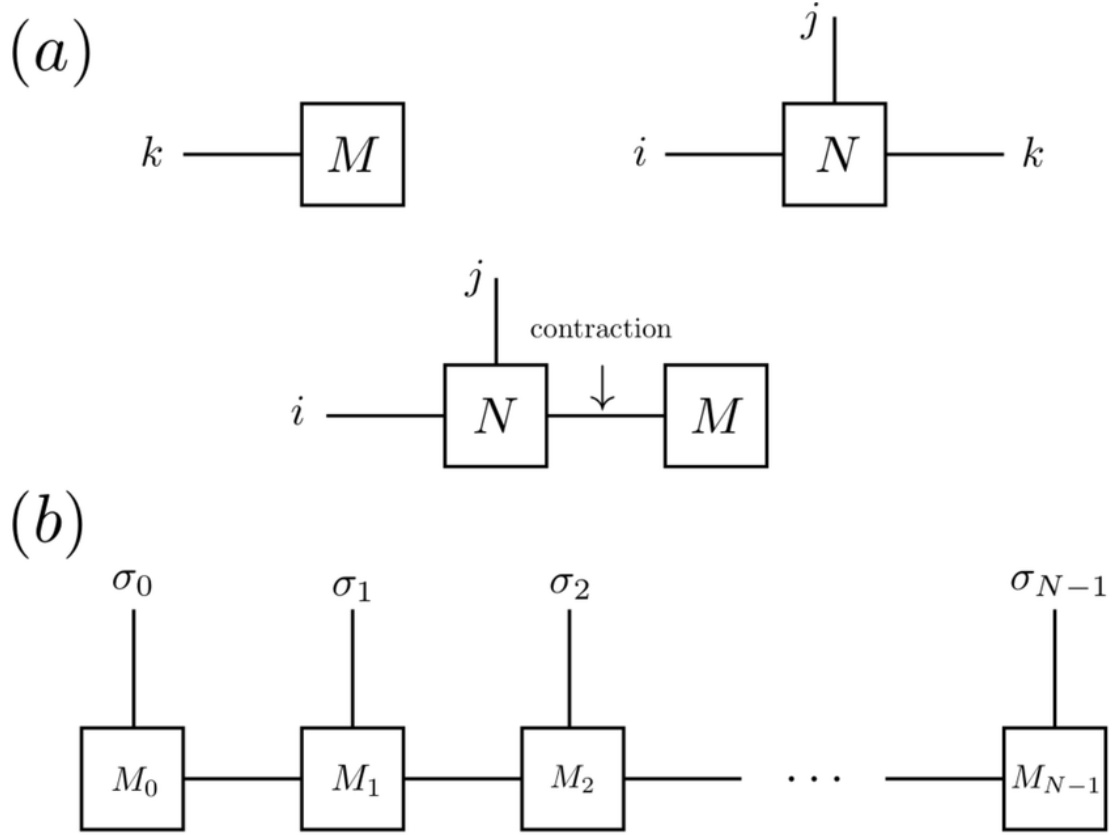


Figure 2.2: (a), on the top, displays a vector, M , and a rank-3 tensor, N , contracted across their common leg k ; (b), on the bottom, displays a complex network of N tensors chained in a tensor train, commonly known as a Matrix Product State (MPS)

indices, which do not participate in contractions, are drawn as dangling legs extending from the network.

Consider a simple example: contracting two tensors $\mathcal{A}_{ij}$ and $\mathcal{B}_{jk}$ to produce $\mathcal{C}_{ik}$. In tensor network notation, $\mathcal{A}$ appears as a node with two legs (for indices i and j), and $\mathcal{B}$ appears as a separate node with two legs (for indices j and k). The leg corresponding to index j in $\mathcal{A}$ is connected to the leg corresponding to index j in $\mathcal{B}$, indicating that these indices are contracted. The remaining legs (corresponding to i and k) dangle freely, representing the free indices of the result tensor $\mathcal{C}$. Figure 2.2 illustrates this concept with more examples.

More complex tensor networks can compactly represent intricate contraction patterns

that would be cumbersome to express algebraically. The graphical representation immediately reveals the structure of the network i.e., which tensors share indices, the connectivity pattern, and which indices remain uncontracted. This makes tensor network diagrams important for visualizing and reasoning about large-scale contractions [2].

2.2 Basic Linear Algebra Subprograms

Basic Linear Algebra Subprograms (BLAS) is a standardized specification for low-level linear algebra operations that serves as a foundation for high-performance numerical computing. Originally developed in the late 1970s [13], BLAS provides a portable and efficient interface for fundamental vector and matrix operations, enabling libraries and applications to achieve near-optimal performance across diverse hardware architectures.

BLAS operations are organized into three hierarchical levels, each characterized by the computational intensity (ratio of arithmetic operations to memory accesses) and scope of the operations:

2.2.1 Level 1: Vector Operations

Level 1 BLAS concerns with scalar and vector operations with computational complexity $O(n)$, where n is the vector length. These operations include vector addition, scalar multiplication, dot products, and norm calculations. Due to their linear computational intensity and high memory-to-computation ratio, Level 1 operations are memory-bound and exhibit poor cache utilization. Examples include AXPY (scaled vector addition) (2.3), DOT (dot product), and NRM2 (Euclidean norm).

$$y \leftarrow \alpha x + y \tag{2.3}$$

2.2.2 Level 2: Matrix-Vector Operations

Level 2 BLAS optimizes matrix-vector operations with computational complexity $O(n^2)$, including matrix-vector products, rank-one updates, and triangular system solves. These operations have moderate computational intensity and remain memory-bound on most modern architectures. Common Level 2 operations include GEMV (general matrix-vector multiply) (2.4) and GER (general rank-one update).

$$y \leftarrow \alpha Ax + \beta y \tag{2.4}$$

2.2.3 Level 3: Matrix-Matrix Operations

Level 3 BLAS covers matrix-matrix operations with computational complexity $O(n^3)$, such as matrix multiplication and triangular solves. These operations exhibit high computational intensity and can achieve efficient cache reuse through careful data blocking and tiling strategies. Level 3 operations are typically compute-bound on modern hardware, making them suitable targets for optimization and parallelization. The general matrix-matrix multiplication (GEMM) (2.5) operation is the cornerstone of Level 3 BLAS.

$$C \leftarrow \alpha AB + \beta C \tag{2.5}$$

Efficient implementations of BLAS, such as ATLAS, OpenBLAS, and Intel MKL, are critical for achieving high performance in scientific computing and machine learning applications [8]. In the context of tensor contractions, BLAS operations, particularly Level 3 GEMM, serve as the primary computational kernel when contractions are transformed into matrix multiplications.

2.3 High-performance Matrix-Matrix Multiplication Implementations

The work presented uses the Tensor Contraction Code Generator (TCCG) [21] library as its high-performance matrix-matrix multiplication engine. TCCG is designed specifically to deliver efficient GEMM kernels across modern CPU architectures by combining the algorithmic principles of blocked matrix multiplication with architecture-aware microkernel generation.

At the highest level, TCCG decomposes the computation into a hierarchy of tiles. The input matrices are partitioned into blocks that fit into the different levels of the processor cache, which reduces main-memory traffic and improves data reuse. The blocked structure also enables the library to expose parallelism naturally: independent blocks can be computed concurrently across CPU cores, while each core operates on cache-resident panels of the matrices.

A central feature of TCCG is its use of a carefully tuned microkernel. The microkernel computes a small block of the result matrix using registers and vector instructions, often with explicit loop unrolling and packing of input matrix panels. This organization minimizes the number of memory accesses per floating-point operation, which is essential for reaching high fractions of peak performance on modern processors.

The library also supports explicit matrix packing, where subblocks of the input matrices are copied into contiguous, cache-friendly buffers before the inner product is

evaluated. Packing eliminates irregular memory access patterns that occur in standard row-major or column-major layouts, and it allows the microkernel to consume data in a streaming fashion. In TCCG, the packed panels are chosen to align with the target architecture’s cache line size and vector width.

Beyond blocking and packing, TCCG implements an adaptive blocking strategy for common matrix shapes. For square matrices, the blocking factors are selected to balance between the L1 / L2 cache capacities and the cost of data movement. For rectangular matrices, the library selects block sizes to maintain high utilization of the inner microkernel while avoiding excessive padding or wasted computation.

It is important to emphasize that TCCG is not just treated as a black-box library, but integrated into the overall implementation. Its performance model is based on the same principles discussed earlier in this chapter: increasing arithmetic intensity, maximizing cache reuse, and exposing parallelism through block-level decomposition. When tensor contractions are mapped to matrix-matrix multiplications, TCCG provides the low-level kernel that performs these multiplications efficiently, making the overall tensor contraction implementation practical for large-scale problems.

2.4 High-performance Tensor Contraction Implementations

The main tensor transposition engine used in this work is the High-Performance Tensor Transposition (HPTT) library [23]. HPTT addresses a data-movement-centric bottleneck that is ubiquitous in tensor-based codes. While a single GEMM call can be made very fast, the full contraction pipeline often requires rearranging tensor data, and inefficient transpositions can dominate runtime if not handled carefully.

HPTT implements multidimensional, out-of-place tensor transpositions using the same high-performance principles introduced earlier for matrix multiplication: blocking, explicit vectorization, a small hand-tuned micro-kernel, and parallelization. Concretely, HPTT decomposes any higher-order permutation into a nest of loops surrounding a micro-kernel that performs a small, cache-resident element reordering. This decomposition has three practical advantages: (1) the micro-kernel can be hand-vectorized for the target instruction set (e.g., AVX, NEON), (2) blocking at multiple levels improves cache reuse and reduces off-chip bandwidth requirements, and (3) the loop nest exposes both data- and task-level parallelism for multicore execution.

$$B_{\Pi(i_1, i_2, \dots, i_N)} \leftarrow \alpha \times A_{i_1, i_2, \dots, i_N} + \beta \times B_{\Pi(i_1, i_2, \dots, i_N)}$$

Figure 2.3: Illustration of HPTT’s tensor transposition semantics and permutation notation (image source: [springer13/hptt](https://springer13.github.io/hptt/)).

An important engineering feature of HPTT is its optional autotuning framework. At plan creation time, the library can either estimate good implementation parameters from a performance model (fast) or explore a search space of loop orders, blocking factors, and parallelization strategies (measure/patient modes) to find the best-performing schedule for the current tensor shape and hardware. This behavior is analogous to FFTW’s planner [6] and makes HPTT robust in applications where tensor sizes and permutations are only known at runtime.

HPTT also supports explicit matrix/tensor packing and specializes micro-kernels to align with cache line sizes and vector register widths, thereby minimizing irregular memory accesses and enabling streaming consumption of input data. The library exposes simple C, C++, and Python interfaces for creating a transposition plan (with or without autotuning) and executing it, for example see Figure 2.4.

```
#include <hptt.h>

// allocate tensors
float A* = ...
float B* = ...

// specify permutation and size
int dim = 6;
int perm[dim] = {5,2,0,4,1,3};
int size[dim] = {48,28,48,28,28};

// create a plan (shared_ptr)
auto plan = hptt::create_plan( perm, dim,
                               alpha, A, size, NULL,
                               beta, B, NULL,
                               hptt::ESTIMATE, numThreads);

// execute the transposition
plan->execute();
```

Figure 2.4: Example usage of HPTT’s C++ interface from the library’s documentation.

In practice, integrating HPTT as the transposition layer in a tensor contraction pipeline significantly reduces the overall runtime, because the cost of rearranging tensor data becomes comparable, if not superior, to the cost of the high-performance GEMM kernels if not optimized.

3 Related Work

Early work primarily optimized for FLOPs, but it is now well understood that minimizing FLOPs alone can lead to prohibitively large intermediate tensors and thus excessive memory consumption. Therefore, a new set of algorithms have been developed to reduce this effect of the memory. This section reviews these contributions, with a focus on techniques that explicitly reduce memory footprint or data movement.

3.1 Treewidth-Based and Graph-Theoretic Methods

Markov and Shi [15] established a foundational connection between tensor network contraction and graph theory, showing that the complexity of contraction is governed by the treewidth of the underlying graph. Minimizing treewidth implicitly controls the size of intermediate tensors and therefore peak memory usage. This is due to the treewidth bounding the size of the largest intermediate tensor produced during contraction. A contraction with treewidth k guarantees that no intermediate tensor will have more than k indices. This directly translates to memory savings. Moreover, low treewidth enables efficient dynamic programming algorithms that can compute the optimal contraction sequence in time exponential only in treewidth, not in the total number of tensors.

However, exact treewidth minimization is NP-hard [4], motivating heuristic approaches. Subsequent work has leveraged graph partitioning and elimination ordering heuristics to approximate optimal contraction paths. For example, tools such as `quickbb` [7] and `flow-cutter` [10] have been adapted to tensor contraction problems. These approaches reduce peak memory by limiting separator sizes, though they are not explicitly designed for memory movement.

3.2 Hypergraph Partitioning Approaches

More recent work models tensor networks as hypergraphs, enabling more accurate representation of multi-index tensors. Gray and Kourtis [9] introduced a hypergraph-based contraction path optimizer that uses partitioning techniques to minimize both

FLOPs and intermediate tensor sizes. This framework allows explicit incorporation of memory constraints and has been widely adopted in tools such as cotengra.

Hypergraph partitioning methods can be extended to multi-objective optimization, balancing FLOPs and memory. For instance, multi-criteria cost functions penalize large intermediate tensors, effectively reducing peak memory usage. However, these methods typically focus on static path selection and do not account for runtime memory movement.

3.3 Slicing Techniques

Slicing is one of the most widely used methods for reducing peak memory. It involves fixing certain indices to specific values, thereby decomposing a large contraction into multiple smaller subproblems. Each slice can be computed independently, significantly reducing memory requirements at the cost of increased total computation. Figure 3.1 illustrates this process.

This approach has been extensively used in quantum circuit simulation, including in Google’s quantum supremacy experiments [1]. Later work formalized slicing as an optimization problem, selecting indices to slice in order to trade off memory and runtime [9]. In practice, slicing is especially attractive when the memory footprint of the original contraction exceeds available RAM, because it can reduce the size of intermediates by several orders of magnitude.

However, slicing also introduces a trade-off between memory and execution cost that must be carefully managed. In the context of quantum circuit simulation, Google’s quantum supremacy experiment used slicing to reduce the peak memory of a 53-qubit simulation, at the expense of performing many more independent contractions than in the unsliced case [1].

The benefits of slicing are not limited to raw memory reduction. Because individual slices are independent, they expose a high degree of parallelism and can be distributed across processors or compute nodes with minimal synchronization. This property is especially advantageous for out-of-core or distributed execution, where fitting a single slice into local memory or cache can substantially improve throughput. Several works therefore treat slicing as a practical mechanism to scale tensor contraction to larger circuits or networks than would otherwise fit on a single device.

While slicing is a powerful technique for enabling contractions that would otherwise be infeasible due to memory limits, it is most effective when used in combination with path optimization and execution-level strategies. Careful selection of slicing indices and integration with low-level data movement optimization are necessary to realize the potential benefits without incurring prohibitive runtime overhead.

3.4 Recompression and Tensor Decomposition

Another class of methods reduces memory by compressing intermediate tensors. Techniques such as singular value decomposition (SVD) and tensor train (TT) decompositions are used to approximate tensors with lower-rank representations.

For example, methods based on matrix product states (MPS) [20] and tensor trains [16] dynamically truncate intermediate tensors to control memory growth. These approaches are particularly effective in structured problems with low entanglement, but introduce approximation error and may not generalize to arbitrary tensor networks which are the primary focus of this work.

3.5 Memory Movement and Data Locality

While peak memory has received significant attention, memory movement is increasingly recognized as a critical bottleneck. Modern hardware architectures exhibit large disparities between computation speed and memory bandwidth, making data movement a dominant cost.

Efforts to reduce memory movement include blocking and tiling strategies [14, 17], which aim to maximize cache reuse, as well as layout transformations that improve data locality. In the context of tensor contractions, permutation of tensor indices plays a key role in determining memory access patterns.

However, most existing work treats permutations as a secondary concern, focusing primarily on algebraic correctness rather than memory efficiency. This represents a significant gap in the literature.

3.6 Summary and Research Gap

The literature on tensor network contraction reveals two complementary lines of work. One line emphasizes pre-contraction optimization, using graph-theoretic and hypergraph partitioning techniques to choose contraction orders that limit intermediate tensor sizes and reduce peak memory across the execution. The other line emphasizes post-contraction execution, using slicing, recompression, and out-of-core scheduling to cope with cases where available memory is insufficient for a single monolithic contraction. Both approaches have advanced the ability to solve larger networks and higher-dimensional problems, but they also expose an important limitation: the predominant focus has been on peak memory and arithmetic cost, while the costs of data movement, tensor layout, and permutation overhead are still treated as subordinate concerns.

This gap is especially salient in modern high-performance contexts, where memory bandwidth and cache locality often dominate actual runtime. As the introduction of this thesis highlights, a contraction strategy with low theoretical FLOP count can still be impractical if it produces intermediates whose layout requires repeated reshaping and transposition. Existing work on contraction ordering and path selection therefore needs to be complemented by optimization that accounts for the global structure of tensor permutations and their impact on memory movement.

Accordingly, the contribution of this thesis is to bridge the divide between contraction planning and execution-level memory optimization. By treating tensor layout as a first-class optimization variable and by analyzing post-contraction permutations across an entire contraction sequence, this work aims to reduce redundant data transformations, improve locality, and achieve better memory efficiency without sacrificing the benefits of established path optimization techniques. In doing so, it seeks to provide a more holistic foundation for efficient tensor network contraction on modern architectures.

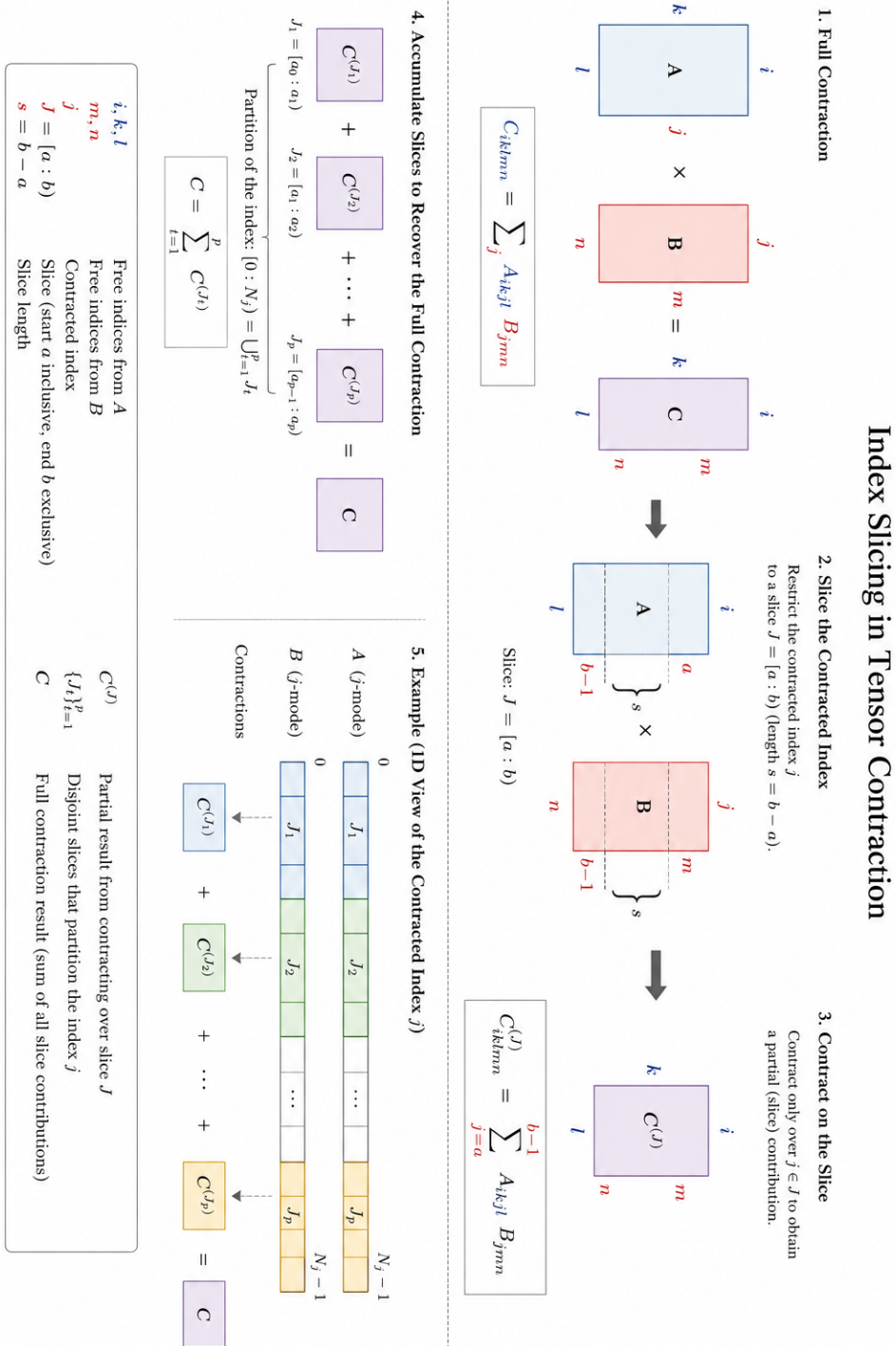


Figure 3.1: Illustration of index slicing in tensor contraction

4 Implementation

The purpose of this chapter is to outline the implementational details of the library developed, the tensor permutation discovery algorithm alongside the required data-structures, and the end-to-end pipeline used for compiling the operations. We provide the reader with a top-down description of our approach, stepping down one abstraction at a time, i.e. from the library’s interface to the core of our algorithm to how it is integrated into a state-of-the-art pipeline.

4.1 Interface

The library exposes a concise Python interface, see Figure 4.1, for permutation strategies that is independent of the concrete optimization algorithm. This interface is defined as a runtime-checkable protocol and makes it possible to register and interchange different strategy implementations without changing the surrounding compilation pipeline.

```
@runtime_checkable
class IStrategy(Protocol):
    @staticmethod
    def find_optimal_permutation(
        network: TensorNetwork, contraction_path: ContractionPath
    ) -> list[TensorOperation]:
        ...
```

Figure 4.1: Outgoing python interface for permutation discovery strategies.

This interface is the library’s public contract for strategy implementations. The `find_optimal_permutation` method is responsible for producing an ordered sequence of tensor operations that realize an optimized contraction plan.

The strategy implementation receives two explicit inputs: a tensor network and a contraction path. The tensor network is represented by the `TensorNetwork` dataclass, see Figure 4.2, which contains the initial tensor collection, the desired output indices, and a size map. The contraction path is a sequence of index pairs that specifies

the pairwise contraction order. Internally, the tensor network manages its tensors through a private `_Tensorpool` abstraction, enabling dynamic reuse of tensor objects across contraction steps instead of allocating repeated tensor storage. This decision is important for performance, and it will be highlighted again in later sections when discussing the algorithm's datastructures and execution behavior. Finally, a strategy's return type is not a raw permutation matrix or a list of index orders, but an explicit intermediate representation in the form of `TensorOperation` objects.

```
@dataclass(init=False)
class TensorNetwork:
    tensors: _TensorPool
    output_indices: list[int]
    size_dict: dict[int, int]
```

Figure 4.2: Snippet of the tensor network dataclass.

The intermediate representation, see Figure 4.3, decouples permutation planning from execution. It allows the optimizer to express both layout transformations and contraction steps as first-class operations that can be later compiled, scheduled, or simulated by the pipeline.

```
class TensorOperation(ABC):
    @abstractmethod
    def apply(
        self, *inputs: TensorOperationResult, **kwargs: dict[str, Any]
    ) -> TensorOperationResult:

class TensorPermutationOperation(TensorOperation):
    ...

class TensorContractionOperation(TensorOperation):
    ...
```

Figure 4.3: Snippets of the intermediate representation objects.

Additionally, strategies expose a `get_peak_memory` method for a second aspect of the interface: cost estimation. It computes the maximum memory footprint that a given path and its permutations will require during execution. Similarly, `get_total_memory` reports the total memory movement associated with the same plan. These metrics are

useful for comparing candidate strategies and selecting one that balances performance and resource usage.

By defining both permutation planning and memory estimation in the same interface, the library provides a coherent abstraction for strategy selection. Specific algorithms, such as the greedy strategy developed in this work, implement this protocol and are therefore interchangeable with alternative strategies that may be added in the future.

4.2 Preparation and Peripherals

4.2.1 Persistent Contraction Path

Before the permutation strategy can be applied, the contraction path must be prepared in a form that preserves the complete tensor network history. The library uses a dedicated `PersistentContractionPath` abstraction for this purpose. This class stores the original contraction path together with a history of tensor network states, and enforces that the history contains exactly one entry for the initial state plus one additional entry for each contraction step.

The persistent path is inspired by persistent data structures such as segment trees, where later versions share unchanged portions of earlier versions through structural sharing. In this implementation, the contraction history is built by simulating each contraction in order and appending only the changed state to the history list while preserving references to unchanged tensors wherever possible. This means that the algorithm can jump directly to any intermediate step without replaying the entire contraction sequence, and that the storage cost grows only with the number of actual changes rather than with a naive full copy of every intermediate network.

The `from_contraction_path` factory method is the entry point for preparing a contraction path. It takes the original tensor network and the proposed path, then constructs the sequence of states needed by the permutation strategy. The history is kept immutable from the strategy's perspective through the use of deep copies when states are retrieved, ensuring that later operations cannot accidentally mutate earlier steps. As a result, both correctness and reproducibility are preserved when the optimizer analyzes different nodes in the contraction tree.

Placeholder for an illustration of the persistent contraction path and state history.

Figure 4.4: Illustration of the persistent contraction path and state history.

This persistent representation is therefore not only a convenience feature, but a core

preparatory step for the algorithm. It is essential for the strategy's ability to inspect the tensor network at arbitrary points in the contraction sequence without recomputing the entire tree, and it ensures that the memory footprint of the preparation phase remains proportional to the number of contracted states rather than to the size of each complete tensor network copy.

4.2.2 APIs and ABIs

The library exposes a dedicated interface module under `tccg_interface`, which acts as the bridge between the tensor contraction logic and the low-level TCCG and runtime. At the highest level, `execute_tccg_contraction` orchestrates the generation of a TCCG description for a given pair of input tensors, discovers the generated function signature, compiles a `pybind11` wrapper, and finally loads and executes the resulting shared object.

This Python-side API hides the details of the TCCG DSL, the C++ code generation pipeline, and the shared library loading semantics behind a single callable contract. The `tccg_interface` package is implemented as a layered ABI stack, with `TCCGGenerator` responsible for producing the `.tccg` description and invoking the installed `tccg` executable, `TCCGDiscoverer` parsing the resulting generated source, `TCCGPyBind11Compiler` compiling the wrapper into a native module, and `TCCGRuntime` loading the compiled module dynamically.

Because the runtime module is created through `pybind11`, the library's ABI is defined by the exact function signature produced by TCCG and exposed through the generated wrapper. The compilation step therefore requires access to the native HPTT headers and libraries, which are discovered from the environment variables `HPTT_ROOT` and `BLIS_ROOT`. The generated TCCG kernels are linked against HPTT to provide the high-performance transpose and packing primitives that are necessary for the matrix-matrix multiplication kernels to achieve practical throughput.

The compiler glue is implemented in `TCCGPyBind11Compiler`, which emits a small C++ wrapper around the generated TCCG function and then invokes the respective C++ compiler with the appropriate include and link paths.

This explicit ABI binding ensures that the tensor contraction kernels generated by TCCG are directly compatible with the HPTT-backed transposition and workspace management used elsewhere in the library. Thus, the `tccg_interface` package serves as the central integration point between the algorithmic layer in Python and the native tensor compute stack defined by TCCG.

Abbreviations

TNC Tensor network contraction

FLOPS floating-point operations

HPC high-performance computing

BLAS Basic Linear Algebra Subprograms

MPS Matrix Product State

GEMM general matrix-matrix multiplication

TCCG Tensor Contraction Code Generator

HPTT High-Performance Tensor Transposition

List of Figures

2.1	Examples of rank-0, 1, 2, 3 tensors (i.e. a scalar, vector, matrix, order-3 tensor)	5
2.2	(a), on the top, displays a vector, M , and a rank-3 tensor, N , contracted across their common leg k ; (b), on the bottom, displays a complex network of N tensors chained in a tensor train, commonly known as a MPS	6
2.3	Illustration of HPTT's tensor transposition semantics and permutation notation (image source: springer13/hptt).	9
2.4	Example usage of HPTT's C++ interface from the library's documentation.	10
3.1	Illustration of index slicing in tensor contraction	15
4.1	Outgoing python interface for permutation discovery strategies.	16
4.2	Snippet of the tensor network dataclass.	17
4.3	Snippets of the intermediate representation objects.	17
4.4	Illustration of the persistent contraction path and state history.	18

List of Tables

Bibliography

- [1] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. Brandao, D. A. Buell, et al. "Quantum supremacy using a programmable superconducting processor." In: *nature* 574.7779 (2019), pp. 505–510.
- [2] J. Biamonte and V. Bergholm. "Tensor networks in a nutshell." In: *arXiv preprint arXiv:1708.00006* (2017).
- [3] J. Brandejs, N. Hörnblad, E. F. Valeev, A. Heinecke, J. Hammond, D. Matthews, and P. Bientinesi. "Tensor Algebra Processing Primitives (TAPP): Towards a Standard for Tensor Operations." In: *arXiv preprint arXiv:2601.07827* (2026).
- [4] E. F. Dumitrescu, A. L. Fisher, T. D. Goodrich, T. S. Humble, B. D. Sullivan, and A. L. Wright. "Benchmarking treewidth as a practical component of tensor network simulations." In: *PloS one* 13.12 (2018), e0207827.
- [5] G. Evenbly and R. N. Pfeifer. "Improving the efficiency of variational tensor network algorithms." In: *Physical Review B* 89.24 (2014), p. 245118.
- [6] M. Frigo and S. G. Johnson. "The Design and Implementation of FFTW3." In: *Proceedings of the IEEE* 93.2 (2005), pp. 216–231. doi: 10.1109/JPROC.2004.840301.
- [7] V. Gogate and R. Dechter. "A complete anytime algorithm for treewidth." In: *arXiv preprint arXiv:1207.4109* (2012).
- [8] K. Goto and R. A. v. d. Geijn. "Anatomy of high-performance matrix multiplication." In: *ACM Transactions on Mathematical Software (TOMS)* 34.3 (2008), pp. 1–25.
- [9] J. Gray and S. Kourtis. "Hyper-optimized tensor network contraction." In: *Quantum* 5 (2021), p. 410.
- [10] M. Hamann and B. Strasser. "Graph bisection with pareto optimization." In: *Journal of Experimental Algorithmics (JEA)* 23 (2018), pp. 1–34.
- [11] S. Hirata. "Tensor contraction engine: Abstraction and automated parallel implementation of configuration-interaction, coupled-cluster, and many-body perturbation theories." In: *The Journal of Physical Chemistry A* 107.46 (2003), pp. 9887–9897.

- [12] C. Huang, F. Zhang, M. Newman, X. Ni, D. Ding, J. Cai, X. Gao, T. Wang, F. Wu, G. Zhang, et al. "Efficient parallelization of tensor network contraction for simulating quantum computation." In: *Nature Computational Science* 1.9 (2021), pp. 578–587.
- [13] C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh. "Basic linear algebra subprograms for Fortran usage." In: *ACM Transactions on Mathematical Software (TOMS)* 5.3 (1979), pp. 308–323.
- [14] R. Li, A. Sukumaran-Rajam, R. Veras, T. M. Low, F. Rastello, A. Rountev, and P. Sadayappan. "Analytical cache modeling and tilesize optimization for tensor contractions." In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2019, pp. 1–13.
- [15] I. L. Markov and Y. Shi. "Simulating quantum computation by contracting tensor networks." In: *SIAM Journal on Computing* 38.3 (2008), pp. 963–981.
- [16] I. V. Oseledets. "Tensor-train decomposition." In: *SIAM Journal on Scientific Computing* 33.5 (2011), pp. 2295–2317.
- [17] N. Park, B. Hong, and V. K. Prasanna. "Tiling, block data layout, and memory hierarchy performance." In: *IEEE Transactions on Parallel and Distributed Systems* 14.7 (2003), pp. 640–654.
- [18] R. N. Pfeifer, J. Haegeman, and F. Verstraete. "Faster identification of optimal contraction sequences for tensor networks." In: *Physical Review E* 90.3 (2014), p. 033315.
- [19] F. Schindler and A. S. Jermyn. "Algorithms for tensor network contraction ordering." In: *Machine Learning: Science and Technology* 1.3 (2020), p. 035001.
- [20] U. Schollwöck. "The density-matrix renormalization group: a short introduction." In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 369.1946 (2011), pp. 2643–2661.
- [21] P. Springer and P. Bientinesi. "Design of a high-performance GEMM-like Tensor-Tensor Multiplication." In: *CoRR* (2016). arXiv: 1607.00145 [quant-ph].
- [22] P. Springer and P. Bientinesi. "Design of a high-performance GEMM-like Tensor-Tensor Multiplication." In: *CoRR abs/1607.00145* (2016). arXiv: 1607.00145.
- [23] P. Springer, T. Su, and P. Bientinesi. "HPTT: a high-performance tensor transposition C++ library." In: June 2017, pp. 56–62. doi: 10.1145/3091966.3091968.